INNOVATEHACK+ 2.0 · GDG ON CAMPUS XAVIER ATENEO

From Idea to MVP

Scoping, Stack Selection, and Building with Google AI Tools

Arron Kian Parejas
AI Engineer

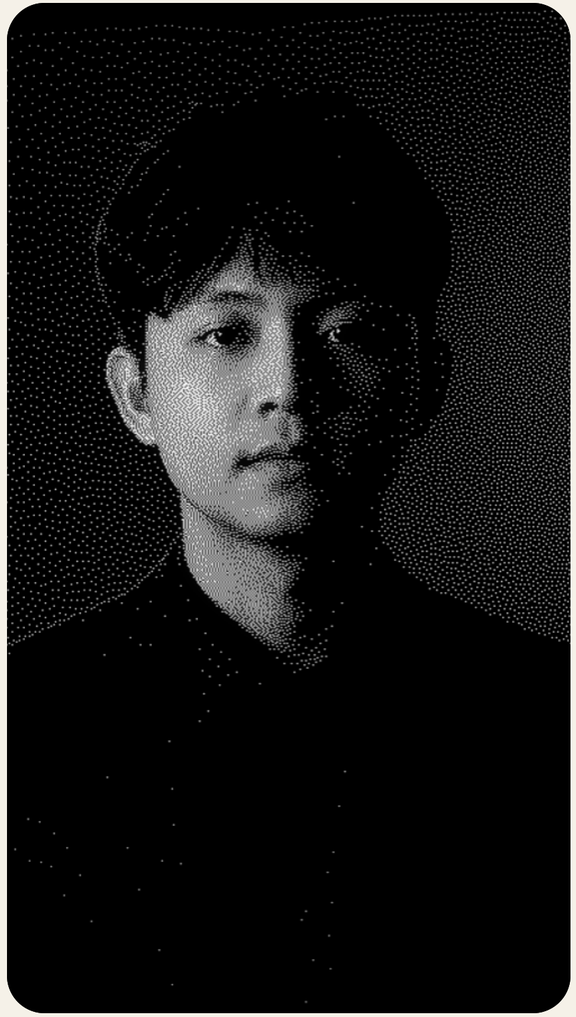
November 16, 2026
Google Meet

45 minutes · 15 slides · bring your own idea



MEET YOUR SPEAKER

A Builder. An Engineer. An Open-Source Contributor.



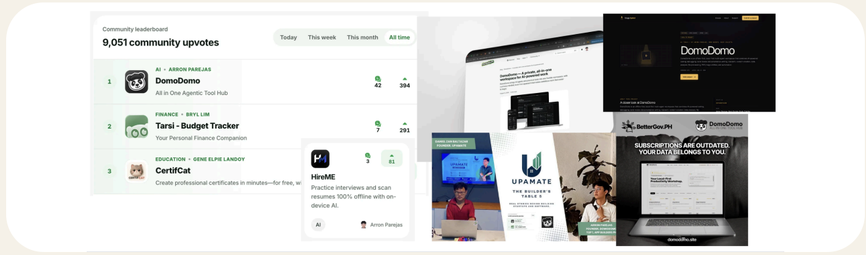
- AI Engineer
- ML Engineering Intern @ FlyRank.ai
- Founder of DomoDomo
- Builder of Helios, an open-source language model
- Former GDG on Campus HAU Chapter Lead
- Open-source contributor
- Ship-A-Little winner, Shipaton 2026

Top 1
AppBuildersPH

Top 7
GitHub · Philippines

7
hackathon wins

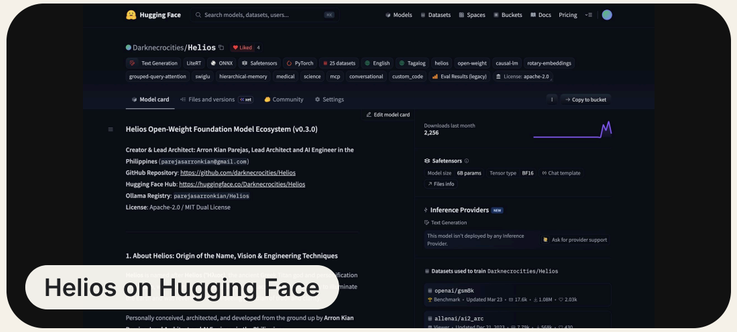
182+
projects











THE HACKATHON MINDSET

Great Ideas Don't Automatically Become Great Products.

01 IDEA → 02 EXECUTION → 03 VALIDATION → 04 PRODUCT

1 **Overengineering**
A perfect system nobody can demo.

2 **Technology before requirements**
Picking the stack before knowing the problem.

3 **Shipping untested code**
Features that fail live on demo day.







START WITH THE PROBLEM

Understand the Problem Before Choosing the Technology.

WHO

Who are we helping?

A specific person, not "everyone".

WHAT

What problem are we solving?

One pain point you can observe.

WHY

Why does it matter?

The cost of leaving it unsolved.

VAGUE IDEA

"Let's build an app for disasters."

- Who? Everyone.
- Problem? "Disasters".
- Why? Because it's a good idea.

→

CLEAR PROBLEM STATEMENT

Barangay residents in flood-prone areas can't report emergencies fast enough, so responders lack accurate locations. We help them send a located report in under 30 seconds.

WHO	Residents of flood-prone barangays
WHAT	A located report in under 30 seconds
WHY	Responders lose time and act on the wrong location

Illustrative example for this talk.

DEFINE YOUR MVP

Build Less. Deliver More.

Emergency reporting platform

Show suggested

Reset

UNSORTED FEATURES (10) — DRAG ONE INTO A BUCKET, OR USE ITS MENU

One-tap emergency report Unsorted

Automatic GPS location Unsorted

Responder dashboard Unsorted

Photo attachment Unsorted

Live report status Unsorted

Push notifications Unsorted

Voice reporting Unsorted

Multi-language support Unsorted

Analytics dashboard Unsorted

Social media sharing Unsorted

Must Have 0

Empty

Should Have 0

Empty

Could Have 0

Empty

Won't Have · Deferred 0

Empty

DESIGN BEFORE YOU CODE

Prototype the Experience First.

01
PROMPT
Describe the screen

→

02
DESIGN
Generate UI options

→

03
PROTOTYPE
Click through the flow

→

04
IMPLEMENT
Build in code

GOOGLE STITCH

From prompt to UI

Turn a text description into screen designs you can iterate on.

stitch.withgoogle.com

MATERIAL DESIGN

Components and tokens

Proven patterns for color, shape, motion, and accessibility.

m3.material.io

GOOGLE FONTS

Type that ships

Free, fast, open-source fonts for web and mobile apps.

fonts.google.com



Report an emergency `Inter · 16/24 · 4.5:1 contrast`

Send report

CHOOSE YOUR TECH STACK

The Right Tool Depends on the Problem.

Try the stack selector (S)

What does your problem need?

Mobile	→	**Flutter** One codebase for Android and iOS, with fast UI iteration.
Backend	→	**Firebase** Managed Auth, Firestore, and Hosting without running servers.
Generative AI	→	**Gemini** Text, image, and structured output via the API or Firebase AI Logic.
Computer Vision	→	**ML Kit** Ready-made on-device APIs for text, barcode, and face detection.
Offline AI	→	**Google AI Edge** Run models on the device with LiteRT, no connection needed.
Custom Infrastructure	→	**Google Cloud** Cloud Run, Vertex AI, and BigQuery for custom services at scale.

THE GOOGLE DEVELOPER TOOLKIT

A Toolkit for Modern Builders.

DESIGN

- Stitch
- Material Design

Describe a screen and get editable designs, then build on proven components.

DEVELOPMENT

Flutter

One codebase for Android, iOS, and web, with a fast UI loop.

BACKEND

Firebase

Auth, database, storage, and functions without running servers.

GENERATIVE AI

- Gemini
- Google AI Studio
- Firebase AI Logic

Prompt, test, and ship generative models from a client or a backend.

MOBILE ML

ML Kit

- Google AI Edge
- AI Edge Gallery
- LiteRT

Ready-made on-device APIs, or run your own model with LiteRT.

CLOUD

Google Cloud

- Cloud Run
- Vertex AI
- BigQuery

Custom services, model training, and analytics at scale.

OFFICIAL DOCS Gemini API · AI Studio · Firebase AI Logic · ML Kit · Google AI Edge · AI Edge Gallery · Flutter · Firebase · Cloud Run · Vertex AI · BigQuery · Stitch · Material Design · Google Fonts

BUILD WITH GOOGLE AI

Give Your Application Intelligence.

01 APPLICATION → 02 AI SERVICE → 03 VALIDATION → 04 RESPONSE

Gemini API

Call Gemini models from your backend.

Docs

Firebase AI Logic

Call Gemini from mobile and web apps.

Docs

Function calling

The model requests; your code acts.

Docs

Structured outputs

JSON that matches your schema.

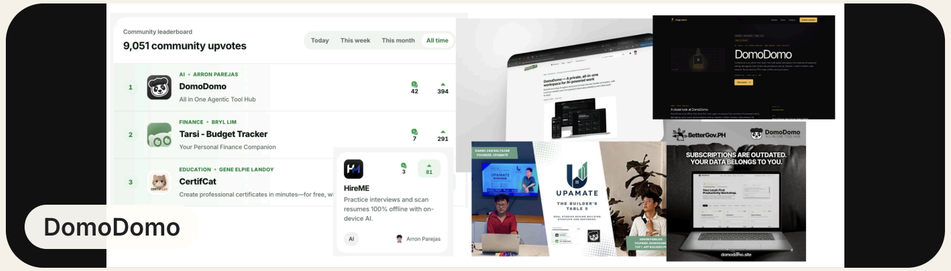
Docs

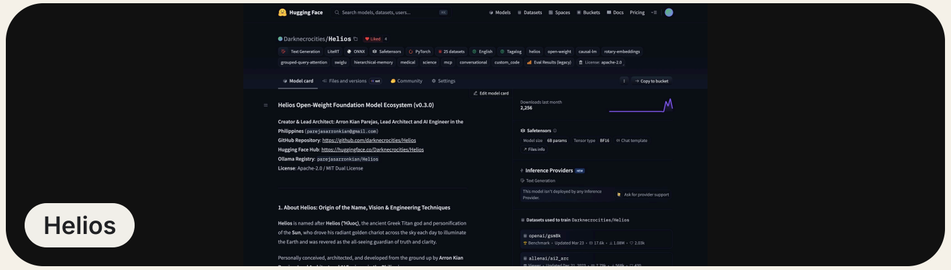
Multimodal AI

Text, images, audio, and video in one request. Always validate what comes back.

Docs

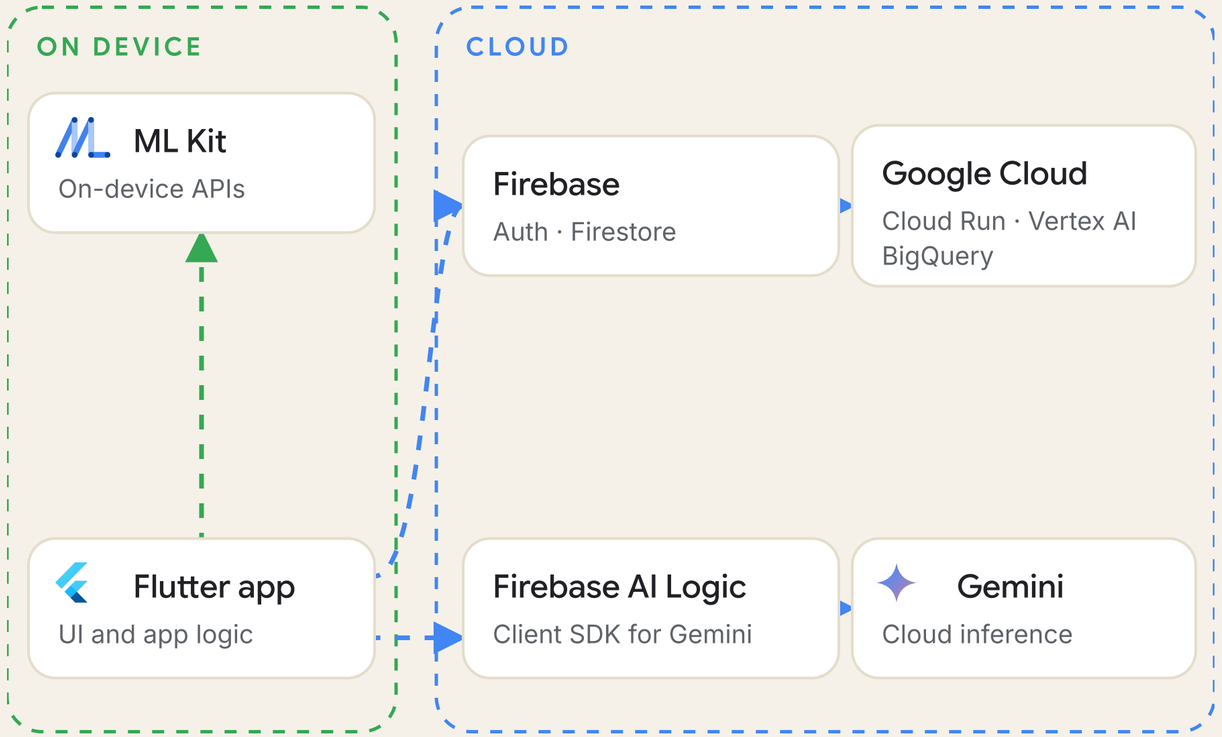






CLOUD AI VS ON-DEVICE AI

Where Should Your AI Run?



	CLOUD	ON-DEVICE
Latency	Network round trip	Immediate, local
Connectivity	Internet required	Works offline
Privacy	Data leaves the device	Data stays local
Constraints	Usage costs and quotas	Memory, battery, model size

● Cloud: Gemini API ● Device: ML Kit · AI Edge · LiteRT

Helios ships LiteRT and ONNX tags on Hugging Face — on-device formats matter for real projects.

Flutter → ML Kit: on-device inference
Flutter → Firebase AI Logic → Gemini
Flutter → Firebase: data + auth
Firebase → Google Cloud: services

PROMPTING IS ENGINEERING

Better Instructions. Better Engineering.

WEAK PROMPT

```
"Build me an AI-powered
application."
```

- × No context, so you get generic code
- × No constraints, so it overengineers
- × No definition of done, so you cannot test it

STRUCTURED PROMPT

```
Context: Flutter app for barangay flood reports.
Constraints: Firebase backend, 3-day sprint,
  works on low-end Android.
Expected outcome: User submits a located
  report in under 30 seconds.
Architecture: Flutter UI, Firestore,
  Gemini via Firebase AI Logic for triage.
Testing: Unit tests for validation,
  one end-to-end submit flow.
```

Illustrative prompt for this talk.

THE CLEAR FRAMEWORK

Structure Your Prompts.

Reveal next (→ / R) | Show all | Reset

C

Context

Who is it for and what exists already?

> Without context, the model guesses at your product.

`Flutter app for barangay flood reports.`

L

Limitations

Time, budget, stack, devices.

> Limits are what stop the output from overengineering.

`3-day sprint. Firebase. Low-end Android.`

E

Expected outcome

A result you can check.

> A checkable outcome is what you can write a test against.

`Located report sent in under 30 seconds.`

A

Architecture

How the pieces connect.

> A diagram beats a paragraph when you mean 'these two'.

`Flutter UI → Firestore → Gemini triage.`

R

Requirements

Tests and quality bars.

> Requirements survive the next person who touches the code.

`Validation unit tests + one end-to-end flow.`

PROMPT SO FAR

`Reveal a letter to start building the prompt.`

SET UP YOUR TEAM

Good Engineering Starts With Good Foundations.

01
GitHub repository
One shared source of truth.

02
Feature branches
One branch per task.

03
Pull requests
Review before merging.

04
Environment variables
Keys stay out of Git.

05
Shared documentation
README, setup, decisions.

06
Task ownership
Every task has one owner.

```
my-mvp/
├── lib/              # app code
├── test/             # unit tests
├── docs/             # decisions, setup
├── .env.example      # names only, no secrets
├── .gitignore        # ignores .env
└── README.md

main          stable, always demo-ready
└ feat/report-form    → PR → review → merge
└ feat/triage-ai      → PR → review → merge

PR checklist
  [ ] branch named feat/…
  [ ] tests pass locally
  [ ] README updated
  [ ] no secrets committed
```

FROM SPRINT TO SHIP

Every Sprint Needs a Deliverable.

01 IDEATION → 02 SETUP → 03 BUILD → 04 TEST → 05 DEPLOY

A three-to-four-day cycle:

DAYS 1–2

Feature development

Build the one core flow end to end.

- Deliverable: one working flow

DAY 2–3

Integration testing

Connect UI, backend, and AI.

- Deliverable: end-to-end run

DAY 3

Bug fixing

Fix what breaks the demo first.

- Deliverable: a green demo

DAY 3–4

Demo preparation

Rehearse, record a backup, deploy.

- Deliverable: deployed and recorded

DEFINITION OF DONE

- One flow you can demo in 60 seconds
- A recorded backup video
- No secrets in the repository
- A README that says how to run it

THANK YOU · Q&A

Build Something That Matters.

- Understand the problem.
- Choose the right tools.
- Engineer the solution.
- Validate everything.
- Ship something that works.

Arron Kian Parejas

Scan for socials and links

linkedin.com/in/arron-parejas

Helios on GitHub

Helios on Hugging Face

domodomo.site